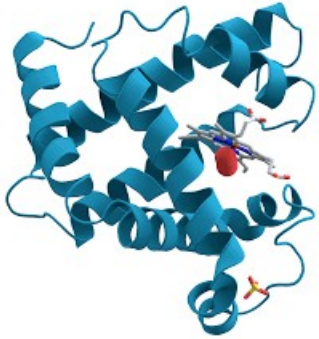
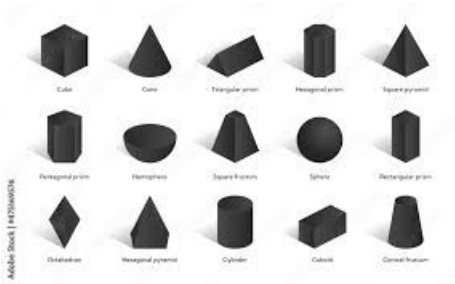
Convulotional Neural Networks (CNNs)

MLPs y su ignorancia de la estructura. Los 4 pilares de las CNN.
Convolución como operación. Arquitectura de las CNN.
Stride y padding. Pooling

¿qué caracteristicas poseen los datos de una imagen?





El problema de los MLPs para imagenes

Los MLPs asumen que las features son independientes y sin estructura. El orden de las entradas no importa, no cambia el modelo

Pero en imágenes y secuencias… Los datos tienen estructura espacial / posicional, Existe correlación local (vecinos importan) El orden sí contiene información

Ejemplo clave En ADN: Cambiar posiciones → rompe motivos biológicos

Necesitamos modelos que: respeten la estructura local, compartan información entre posiciones y generalicen patrones independientemente de su ubicación

Convolutional Neural Networks (CNNs)

Inspiradas por cómo funciona el “cerebro” para ver,

Están diseñadas para capturar “invarianzas” y “equivarianzas” en el tratamiento de imágenes

Son Redes Multicapas poco conectadas y donde las neuronas de la misma capa comparten parámetros.

Entrenar una DNN en imágenes requiere demasiados parámetros: A modo de ejemplo una imagen color 200 × 200 pixels, la 1er capa posee 120,000 Ws y hay que sumar varias capas...

Características de los datos de las imágenes (secuencias?)

Correlación local: la probabilidad de que 2 pixels adyacentes posean el mismo color (o similar) es “muy” superior al azar.

Invarianzas roto-traslacionales. Grupos de pixels que forman un elemento no importa donde estén en la imagen o con qué orientación

Equivarianzas (traslacionales). Un grupo de pixels forma un borde y la presencia del mismo se evidencia independientemente de donde esté localizado en la imagen

¿Cómo aprovecho este conocimiento?

Los 4 conceptos de las CNNs

Jerarquia

Localidad

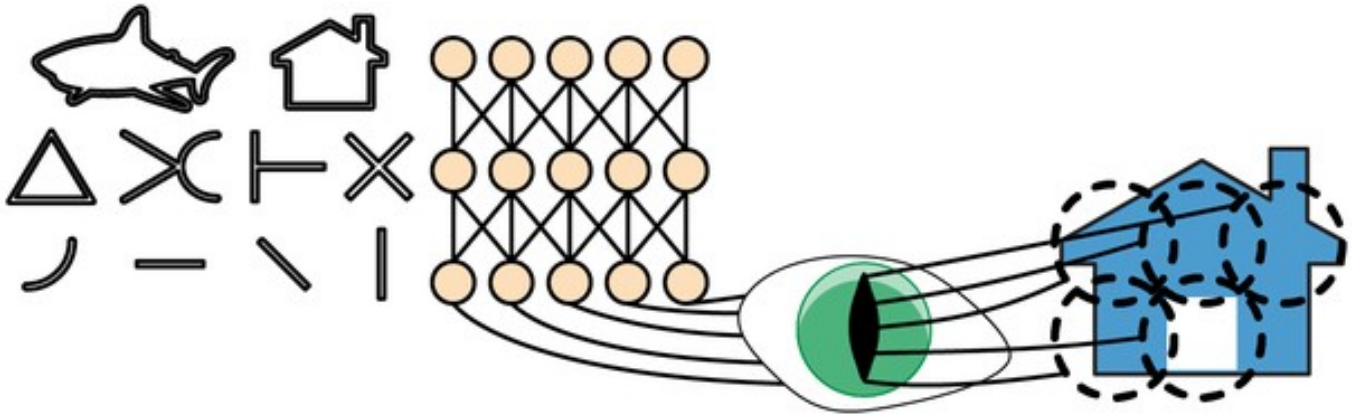
Equivarianza

Invarianza

Jerarquía

Al “subir” en los niveles jerárquicos las capas “aprenden” estructuras (de la imagena) cada vez más complejas

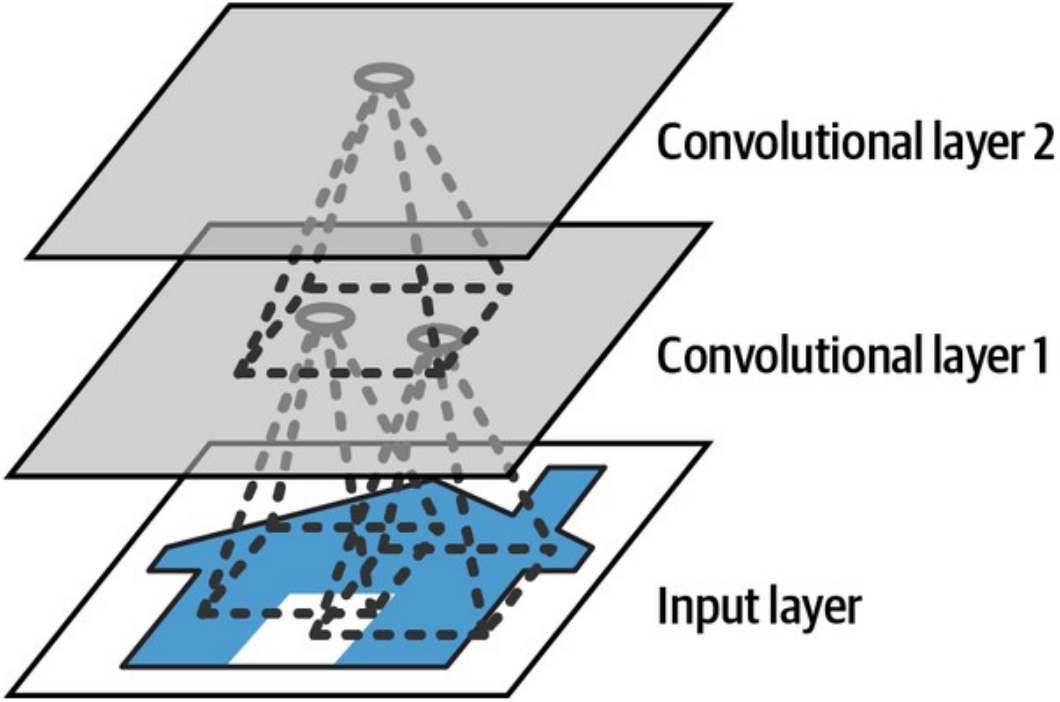
Los niveles bajos detectan bordes, luego formas y asi hasta la comprensiòn completa de la imagen



Localidad

Supongamos que “una neurona” de la primer capa de la red “ve” un grupo de 3x3 pixels (su receptive field) y “aprende” a detectar en ella un borde.

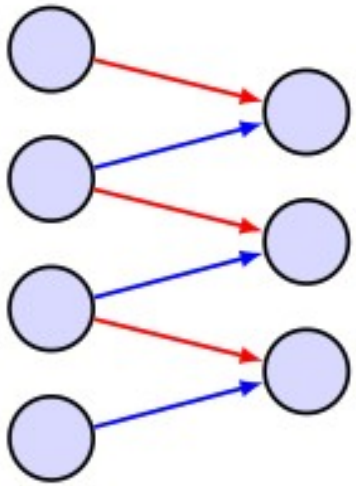
De este modo aprende algo local (el receptive field)



Equivarianza

Supongamos que un grupo de neuronas aprende a ver un cuadrado (o un ojo) en su campo receptivo.
La red debe poder generalizar la identificación del cuadrado en cualquier lugar de la imagen sin que en el conjunto de entrenamiento haya cuadrados en todos los campos receptivos.

Esto se logra replicando los parámetros de las neuronas (W,b) en todas las neuronas equivalentes del campo receptivo. (menos conexiones y menos parámetro)



Invarianza

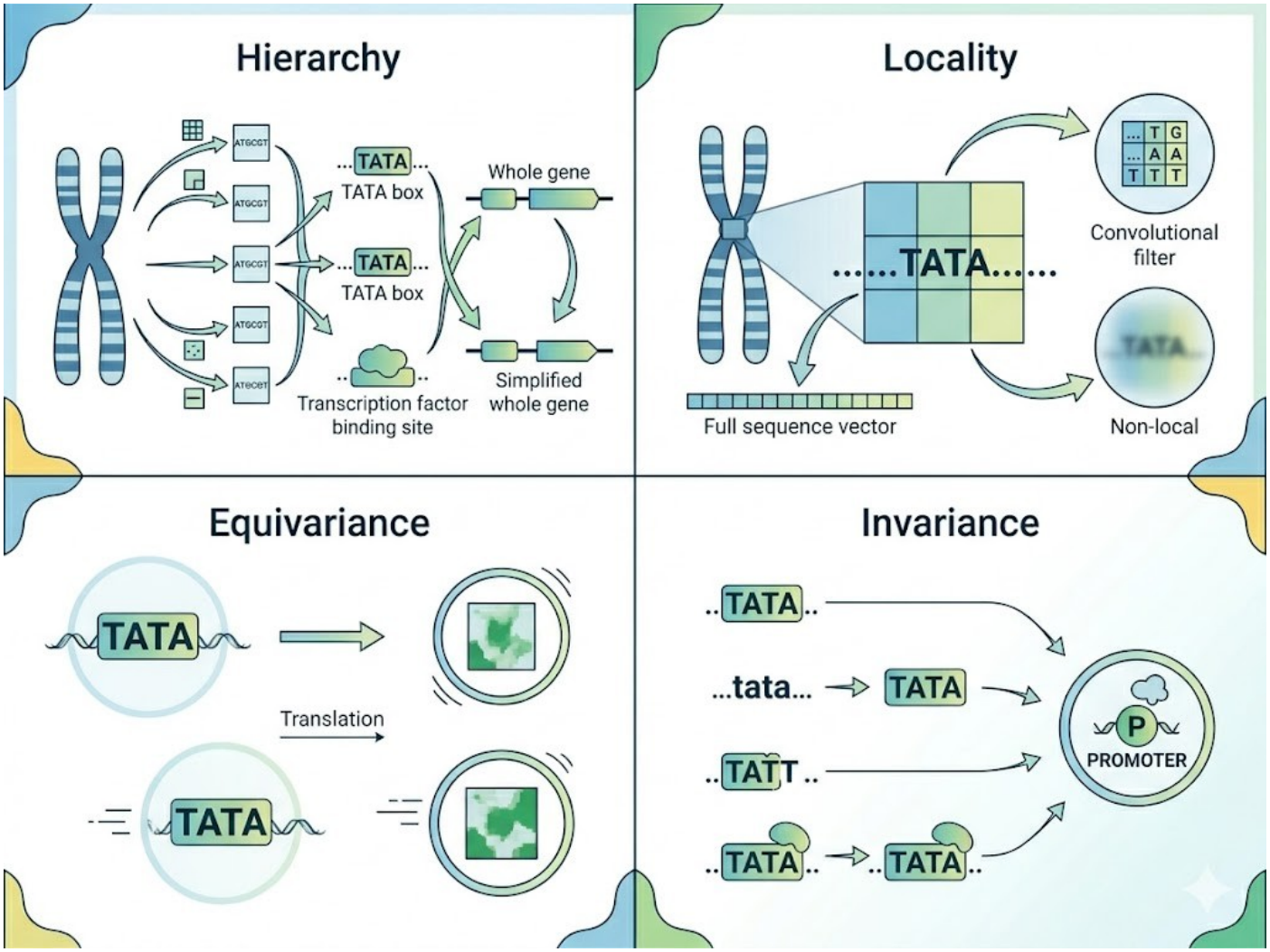
Queremos que la red detecte un patrón independientemente de su posición exacta
Ejemplo:
→ un motivo en ADN o un borde en una imagen
→ debe ser reconocido aunque esté desplazado algunos píxeles/nt

La salida final no cambia (o cambia poco) ante pequeños desplazamientos

Nota:

Equivarianza: la representación se mueve con el patrón
Invarianza: la representación no depende de dónde esté el patrón

En ADN


Hierarchy
ATGCGT
ATGCGT
ATGCGT
ATGCGT
ATGCGT
...TATA...
TATA box
...TATA...
TATA box
Transcription factor binding site
Whole gene
Simplified whole gene
Locality
.....TATA.....
Full sequence vector
... T G
... A A
T T T
Convolutional filter
TATA
Non-local
Equivariance
TATA
Translation
TATA
Invariance
..TATA..
...tata...
TATA
..TATT..
..TATA..
..TATA..
P
PROMOTER

Filtros Convolucionales (o kernels)

El campo receptivo y la equivarianza traslacional da lugar a los “Filtros convolucionales” (filtro = reconocedor de patrones en imágenes)

El “output” del filtro es el “Feature Map”

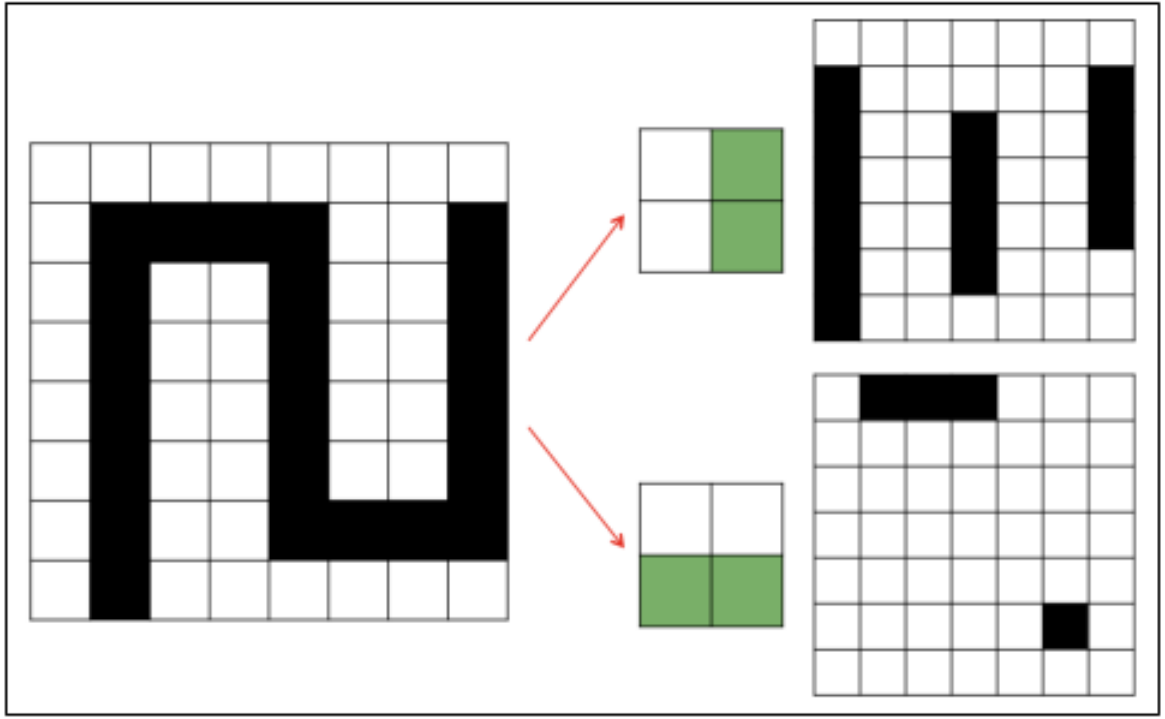


Figure 7-6. Applying filters that detect vertical and horizontal lines on our toy example

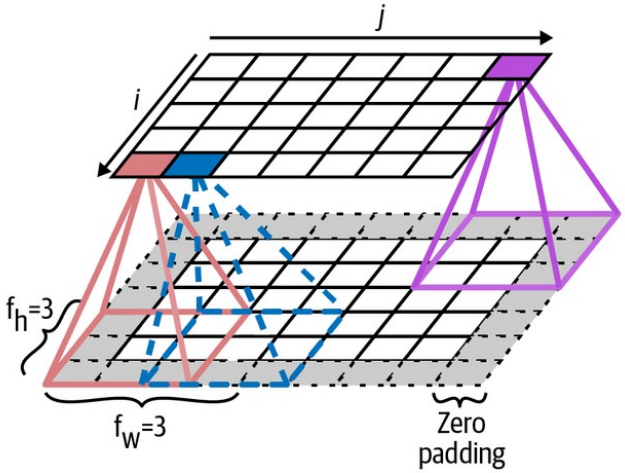
Filtros convolucionales en acción

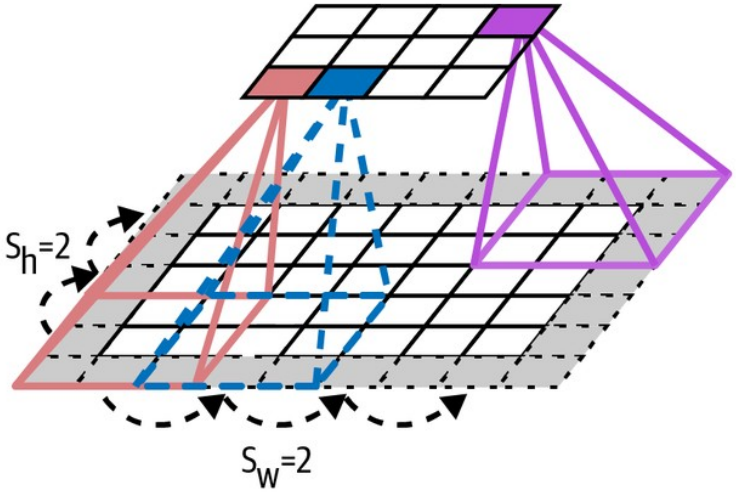
Izquierda: Imagen Original, Centro: Filtro vertical, Derecha Filtro Horizontal

(a) (b) (c)

Stride y Padding

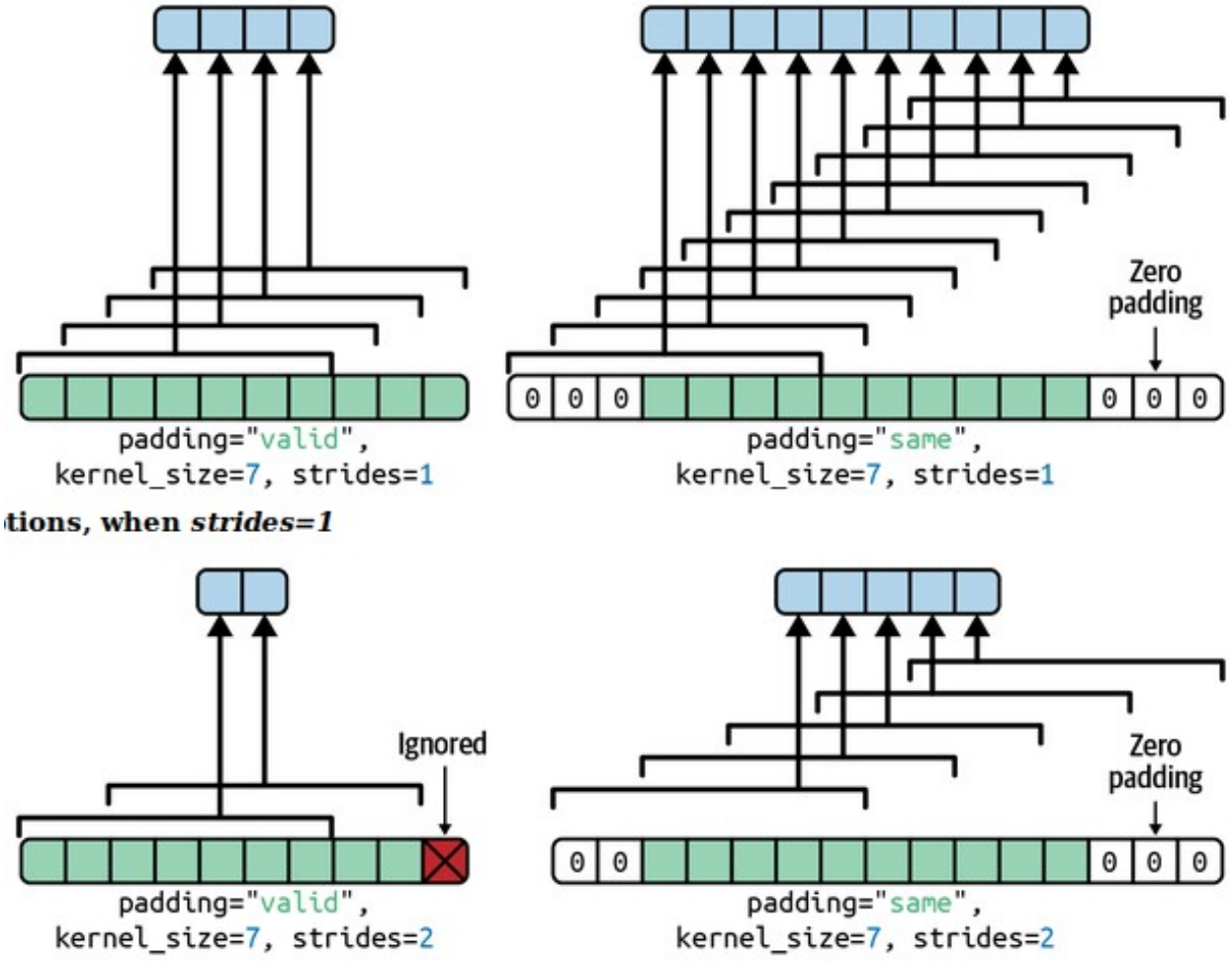
Además de su campo visual definido por d2 (ya que suelen ser cuadrados), los filtros poseen el “stride” (paso) que es cada cuanto se mueve y el padding (como se completan los bordes)





Si stride=1, y hay 0 padding el àrea de salida = area de entrada

Idem en secuencia...



Resumen convolución y filtros

Convolución es un filtro (kernel) sobre el input, que se deplaza (sliding window) y en cada posición:
→ se calcula una combinación lineal (dot product)

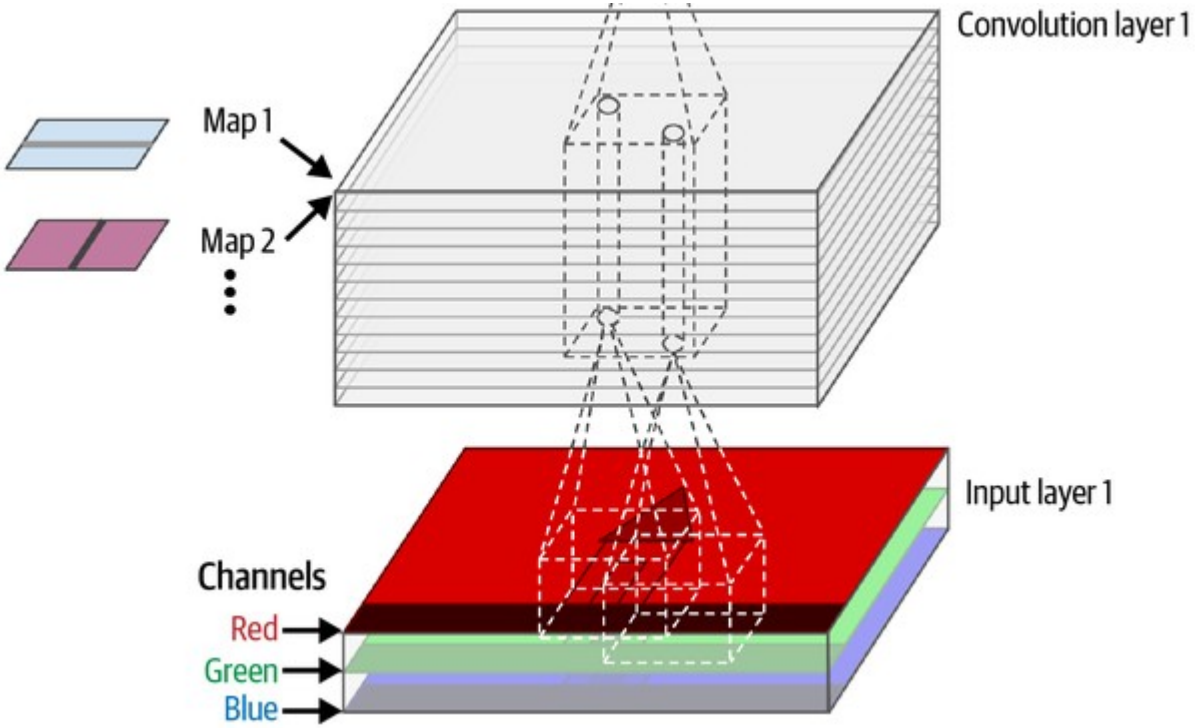
Es un "detector de patrones" (prende a responder a un patrón específico) Si el patrón está presente: → dispara

Al Output del filtro se denomima: Feature Map
Cada valor del FM indica cuánto está presente el patrón en esa posición

En ADN
- El filtro actúa como un detector de motivos (motifs)
  - → sitios de unión de TF
  - → combinaciones específicas de nucleótidos
  - El mismo motivo puede aparecer en distintas posiciones → la convolución lo detecta en todas

Del área a volumen

Los filtros no operan sobre un solo mapa (una capa de input), sino sobre varias capas (o sea un volumen)



El “volumen”

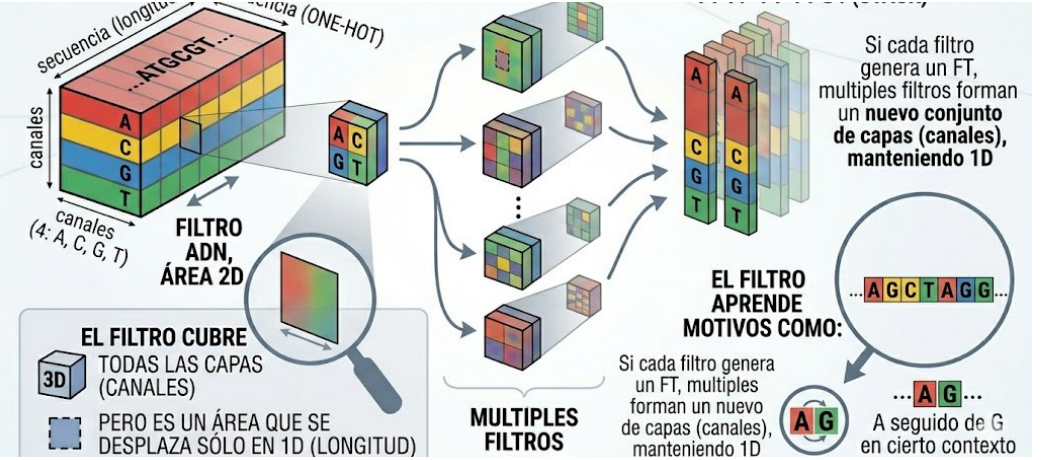
El input no es solo una matriz 2D → es un volumen: alto × ancho × canales. Ejemplos: Imagen color → 3 canales (RGB)

ADN (one-hot) → 4 canales (A, C, G, T) el filtro en estos casos cubre todas las capas (canales) del input, pero solo una región espacial (secuencia) pequeña

Si cada filtro genera un FT, multuples filtros forman un nuevo volumen

En ADN, el filtro Aprende patrones como:

→ “A seguido de G en cierto contexto”



“Tips” filtros

Deben ser pequeños (l=3-5 a veces incluso 1, mayores sólo en las capas iniciales)

Más filtros (capas) pequeños es superior a filtros de mayor tamaño

Usar S=1 y 0 padding permite “mantener” el contenido de información. (para reducirla es mejor usar pooling...)

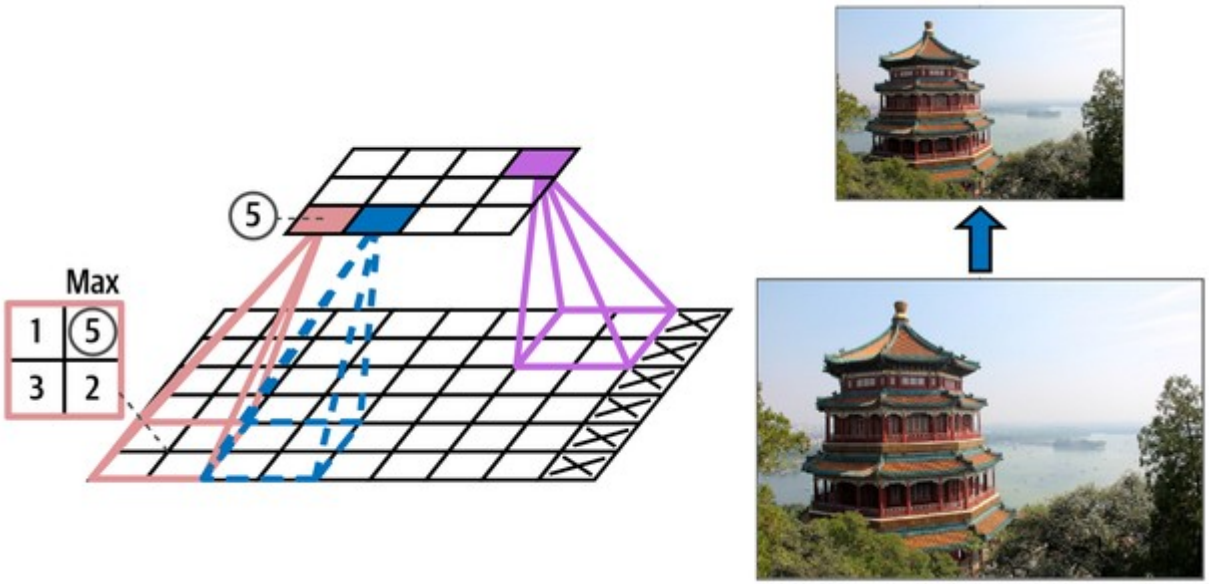
Los filtros NO se crean, surgen del entrenamiento!

Max Pooling

Permite la invarianza traslacional, e incluso que la imagen tenga cierto nivel de ruido.

Y la reducción de la dimensionalidad

Est filtro pasa el valor máximo del campo visual



Ejemplo de Max Pooling

El “max” pooling permite que la red reconozca el patron aún si algunos pixeles son diferentes y/o estan corridos

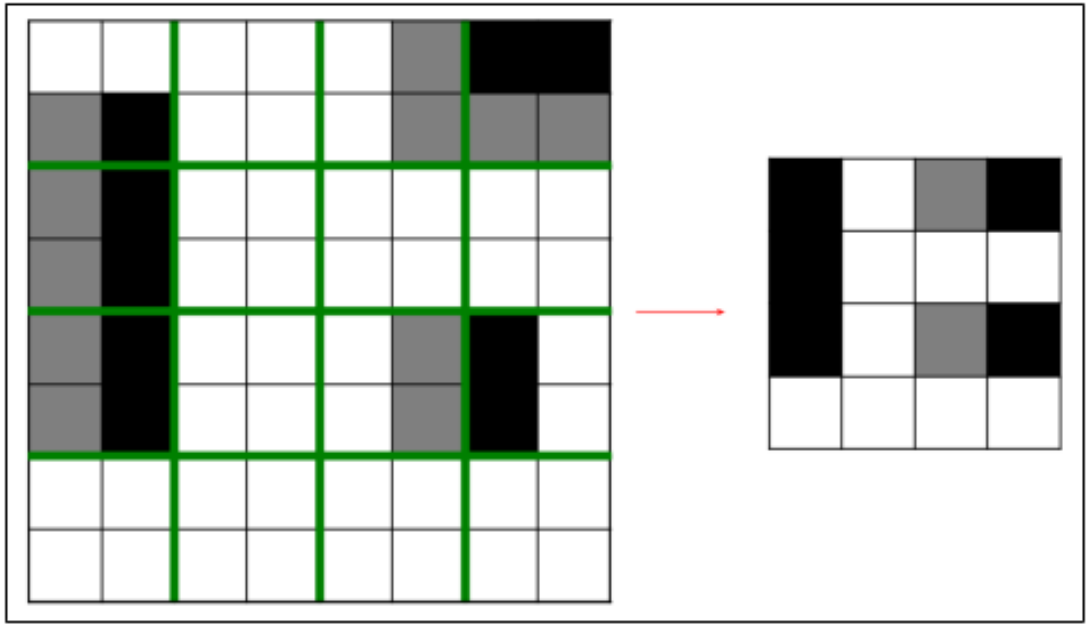


Figure 7-13. Max pooling significantly reduces parameters as we move up the network

Max Pooling

Opera sobre regiones pequeñas (ej: 2×2 o 1D equivalente) Selecciona el valor máximo dentro de cada región

Reduce la dimensión manteniendo la señal más fuerte

¿Qué logra?

Invarianza traslacional (local)→ pequeños desplazamientos no cambian la activación

Robustez al ruido→ ignora variaciones menores

Reducción de dimensionalidad → menos parámetros en capas siguientes

Si un patrón está presente:

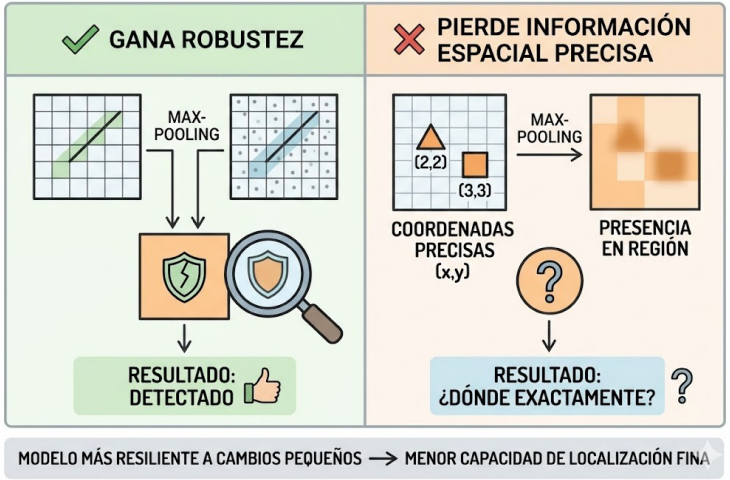
→ no importa exactamente dónde dentro de la región

→ el max pooling lo preserva

El Max-pooling posee un claro Trade-off (clave)

Gana robustez

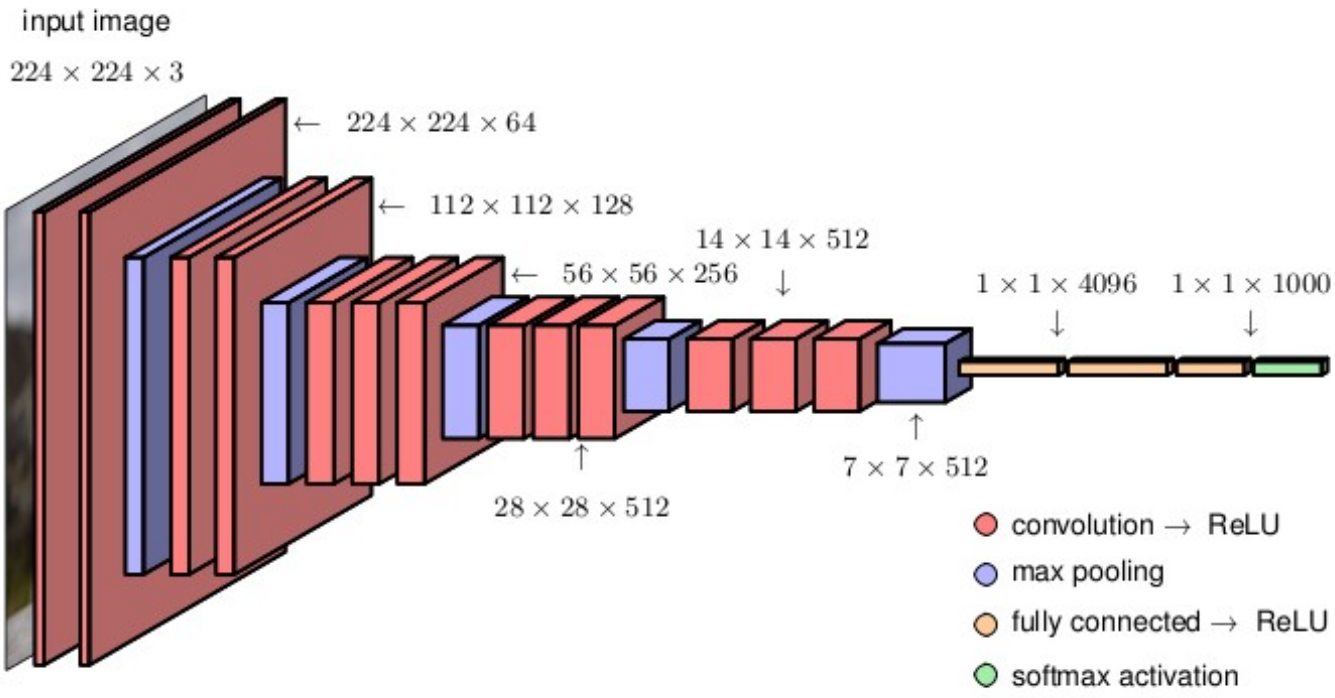
Pierde información espacial precisa



Max pooling Imágenes vs secuencias (ADN)

- En imágenes: → suele ser muy efectivo
- En secuencias de ADN:
- → puede perder información relevante
- → la posición exacta del motivo puede importar
- No siempre mejora la performance

CNN completa



Resumen conceptual

Una CNN aprende detectores de patrones locales → filtros convolucionales
Estos detectores se aplican en toda la imagen/secuencia → compartición de parámetros

¿Qué construye la red?
Capas bajas:
→ patrones simples (bordes / motivos)
Capas altas:
→ combinaciones más complejas
La red construye una representación jerárquica del dato

¿Por qué funciona? - Localidad, Equivarianza, Invarianza, Jerarquía

En secuencias de ADN Los filtros aprenden: → motivos relevantes (ej: sitios de unión de TF)

Preguntas a ver si entendimos

¿Por qué un MLP no es adecuado para trabajar con imágenes o secuencias como ADN?

¿Qué significa que un modelo sea “equivariante traslacionalmente”? y
¿Cómo se logra esto en una CNN?

¿Cuál es la diferencia conceptual entre equivarianza e invarianza?

¿Qué aprende realmente un filtro convolucional?

¿Qué efecto tiene aumentar el stride o aplicar pooling sobre la información del input?
¿Qué se gana y qué se pierde?

Y ahora el TP

Objetivo: El objetivo del presente TP es familiarizarse con las CNNs, particularmente sus dos componentes principales los “convolutional filters” y las capas de “pooling”. Para ello avanzaremos en el aǹalisis de secuencias buscando predecir la unión de un factor de transcripciòn (JunD) al ADN.

En este caso el archivo que bajamos de encode: jun_np_chr22_GRCh38.bed , posee los picos para el cromosoma 22. A continuación se muestran las 2 primeras líneas del mismo

chrom	pos_i	pos_f	.	1000	.	val	-1	nada	size
chr22	20777767	20778110	.	1000	.	363.72008	-1	3.79036	178

Las primeras tres columnas son obvias, luego sigue un “.”, que al igual que el otro “.” el “-” y la columna que bautice “nada” no se utilizan en un BED de este tipo. El valor de 1000 es la columna que utilizan los Geonome Browsers para ver el pico (y tampoco nos importa) y el valor que bautizamos “val” es la INTENSIDAD el pico (o sea la señal)
Por otro lado tenemos la secuencia del Chr22 del Genoma de ref version GRCh38 en formato fasta para poder obtener las secuencias que reconoce el TF.

El resultado del ChiPSeq... archivo BED

Ejercicio 5.1 (opcional). Escriba un código que permita a partir del BED y el fasta obtener las secuencias donde el FT se une al ADN (tambièn puede obtenerlos con samtools). Analicè además la distribución de intensidad de los picos del BED y del tamaño de la secuencia reportada. Vaya a alguno de los genome browsers cargue el BED y vea si puede visualizar en el mismo los picos.

Ejercicio 5.2: a) genere los 3 sets de secuencias negativas. Para el conjunto N3 (secuencias del genoma que NO unen el FT) puede en primer lugar tomar secuencias (de largo fijo) adyacentes y/o a “n” nt de distancia del sitio de unión del FP. Alternativamente puede seleccionar secuencias del Chr22 al azar (o puede probar ambas).
b) escriba el codigo para a partir del fasta con las secuencias de ADN las convierta en un tensor de pytorch de dimension **Nseqs, 4, largo Seq ¿Que cambia aca respecto del TP previo?**

c) integre al código “b” la opciòn de hacer “0” padding y/o recorte las secuencias a un largo fijo.
d) Escriba el codigo que genera además el tensor que posee la clasificaciòn real de la secuencia en si une (y_true=1) (o no, y_true=0) el FT. Usualmente a este tensor se lo llama y_true.
Nota: recomendamos manejar los datos en un DF que permita además de realizar las conversiones mencionadas separar el dataset en entrenamiento y validación.

1er CNN

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class DNA_CNN(nn.Module):
   def __init__(self):
       super(DNA_CNN, self).__init__()

            c=8
       k=5
       self.conv1 = nn.Conv1d(in_channels=4, out_channels=c, kernel_size=k)
       # Fully connected layer: Maps to the output
       # Calculate the output size after the convolutional layer
       conv_output_size = c * (nt_len - k + 1)  # (L - K + 1) formula

       self.fc1 = nn.Linear(conv_output_size, 1)  # Maps to 1 output (binary classification)
```

El fwd

```
def forward(self, x):
    # Apply convolution, followed by ReLU activation
    x = F.relu(self.conv1(x))

    # Flatten the output from the convolutional layer
    x = x.view(x.size(0), -1)  # Reshape to (batch_size, flattened_features)

    # Fully connected layer and output
    x = self.fc1(x)
    x = torch.sigmoid(x)  # Apply sigmoid for binary classification
    return x
```

5.3 Armando la CNN

Ejercicio 5.3 Pruebe que el codigo y su OHE “funcionan” inicializando la red y pidiendo que procese un batch de input

```
my_nn = DNA_CNN()

output = my_nn(test)
```

Analice brevemente la salida (en cuanto a valores, dimension)

Si quiere, para tener como referencia, aproveche y calcule los parámetros de calidad del modelo, como ser la “Confusion Matrix”, la Accuracy (o el F1-Score) y las curvas ROC / PR y su AUC.

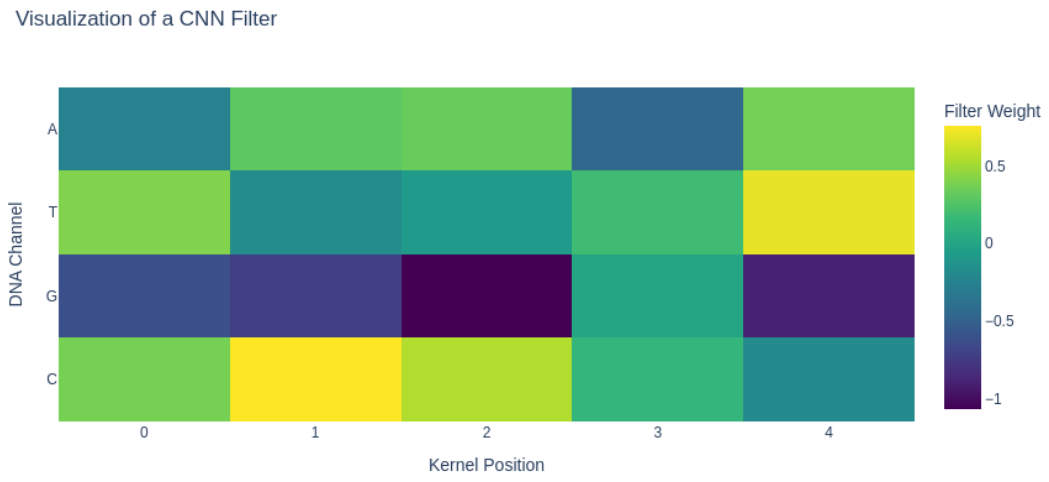
Entrenamiento y primeras CNN

Ejercicio 5.4 Utilizando los parámetros sugeridos (#filtros 8, dimesión 5 y con secuencias de 100nt), realice el entrenamiento de la DNA-CNN. Obtenga y compare para el modelo no entrenado, y para su entrenamiento en 10, 50 y 100 epocas, la LF, la matriz de confusión, la Accuracy, el F1-score y las curvas ROC y PR junto a su AUC. Compare si todas las métricas siguen la misma tendencia (o no).

```
my_nn = DNA_CNN()
criterion = nn.BCELoss()  # Binary Cross Entropy Loss
optimizer = optim.Adam(my_nn.parameters(), lr=0.001)
dataset = TensorDataset(train, y_true)
dataloader = DataLoader(dataset, batch_size=10, shuffle=True)

for epoch in range(epochs):
   total_loss = 0
   for batch_X, batch_y in dataloader:
       output = my_nn(batch_X)  # Predict the output
       loss = criterion(output.squeeze(), batch_y.float())
       optimizer.zero_grad()  # Zero the gradients
       loss.backward()        # Compute gradients
       optimizer.step()       # Update weights
```

Entrenamiento y primeras CNN

Ejercicio 5.5. a) Explore como se modifica la performance de la red al variar el número de filtros (valores sugeridos 4,8, 12 y 16) y su dimensión (K=3, 5, 10, 15). Analice los resultados.
b) Elija un tamaño de filtro, y visualice los filtros entrenados, seleccione un filtro y analicé el resultado.



Max Pooling

Ejercicio 5.6. Agregue a su DNA_CNN una poolling layer y analice el efecto de la misma en la performance del modelo. Pruebe variar el poolin_size y el stride.
A diferencia de los modelos para reconocer imágenes donde el pooling puede ser sumamente efectivo, en ADN no se ha visto que el mismo lo sea ¿que le muestran sus datos?

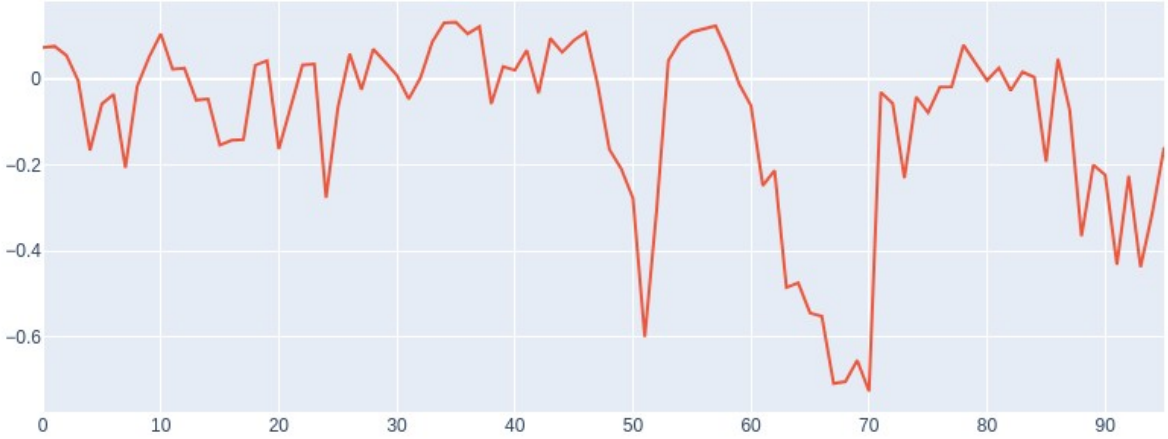
```
        k = 5  # Kernel size for the convolutional layer
        pool_size = 2  # Kernel size for max pooling
        self.conv1 = nn.Conv1d(in_channels=4, out_channels=c, kernel_size=k)
        self.pool = nn.MaxPool1d(kernel_size=pool_size)  # Max pooling layer

        conv_output_length = (100 - k + 1)  # Length after convolution (L - K + 1)
        pooled_output_length = conv_output_length // pool_size  # Length after pooling

        conv_output_size = c * pooled_output_length  # Total flattened size
        self.fc1 = nn.Linear(conv_output_size, 1)  # Fully connected layer

def forward(self, x):
        x = F.relu(self.conv1(x))  # Convolution + ReLU activation
        x = self.pool(x)  # Apply max pooling
        x = x.view(x.size(0), -1)  # Reshape to (batch_size, flattened_features)
```

MASKING

Ejercicio 5.7. Seleccione alguna (la mejor?) de las redes entrenadas en los ejercicios previos. a) obtenga la prediccion para todas las secuencias TP del Chr22, compare el score de las predicciones de cada secuencia con la intensidad del pico correspondiente en el BED. ¿Que le parece los resultados?

b) Escriba un codigo que permita enmascarar regiones de la secuencia y evaluar (como en el ejemplo) el impacto del mismo sobre la prediccion de la red. Utilice esta herramienta para seleccionar las regiones de 10-25nt de longitud de cada secuencia que son màs importantes para el modelo.

c) Obtenga las regiones mas relevantes y realice un MSA (puede utilizar un programa externo online o aprovechar sus conocimientos de BioinfoAv para hacerlo en python). Luego obtenga y analice el logo del MSA correspondiente.

d) (opcional) Realice el análisis de secuencias relevantes-MSA-logo utilizando una red con pocos filtros (4?) y un kernel de imension similar al de las secuencias a extraer (i.e 5-15nt), compare los parámetros del filtro con el logo. ¿Coinciden?

Masking =>